



Loops And Repetitive Work

Assoc. Prof. Dr. Kadir DEMİR

Res. Asst. Mehmethan ERDURAN

This Week In Python

- 1 Choose for or while for a task
- 2 Use an accumulator
- 3 Avoid common loop boundary mistakes

Seven days of sales arrive one by one. What should the weekly report show?

What would the program need to know?

A Loop Repeats A Specific Job

Choose the loop from what you know.

1. Use for when the number of repetitions is known.
2. Use while when a condition decides when to stop.
3. Make sure a while condition can change.

Which loop fits seven daily sales values?

Count With range()

The stop value is excluded.

```
for day in range(1, 8):  
    print(f"Day {day}")  
print("Finished")
```

How many times does the body run?

Accumulators Hold Running Results

Start the total before the loop.

1. Read one value in each repetition.
2. Add it to the running total.
3. Divide after the loop for an average.

Example: [examples/02_accumulator.py](#)

Add Daily Sales

The running total changes once per day.

```
total = 0.0
for day in range(1, 4):
    sale = float(input(f"Day {day}: "))
    total += sale
print(f"Total: {total:.2f} TRY")
```

Trace total for 100, 200 and 50.

while Needs An Exit

Every repetition should move toward stopping.

1. Check the condition before each repetition.
2. Update the value used by the condition.
3. Use break for an explicit exit when appropriate.

Press Control+C if a demo loop does not end.

Repeat Until input Is Valid

The prompt appears again for invalid input.

```
quantity = int(input("Positive quantity: "))
while quantity <= 0:
    print("Try again")
    quantity = int(input("Positive quantity: "))
print("Accepted:", quantity)
```

Try 0, then -2, then 5.

Spot The Off-By-One Error

`range(1, 7)` excludes 7.

```
total = 0
for day in range(1, 7):
    total += int(input(f"Day {day}: "))
print(total)
```

The task needs seven days. Correct the loop.

Live Coding: Sales Monitor

Collect seven values and report a result.

1. Use a loop to ask for each day.
2. Calculate total, average and days meeting a target.
3. Reject a negative sale and ask again.

Test seven zeros and then a mixed week.

Project A: Daily Sales Tracker

Report total, average, minimum and target count.

projects/project_a_sales_tracker.py

Trace how highest changes.

Project B: Service Queue Simulator

Use commands to add and serve customers.

`projects/project_b_queue_simulator.py`

Explain why the loop eventually stops.

In-Class Lab: Weekly Sales Monitor

Your Challenge

Assignment: `week04/lab-quiz/lab_assignment.md`

Submit in your repository: `week04/lab-quiz/`

Build it. Check a boundary. Explain one decision.

GitHub Checkpoint

Lab and quiz: week04/lab-quiz/

Keep weekly homework in week04/ outside that folder.

```
git add week04/lab-quiz/ README.md  
git commit -m "week04: complete lab quiz"  
git push
```

Show the lab, one test and the commit.

See You Next Week